EXPERIMENT-6

AIM:

To write a Program to implement error detection and correction using HAMMING code concept, and make a test run to input data stream and verify error correction feature.

CODE:

```
def redundant_bits(m):
    r=0
    while 2**r < m+r+1:
        r+=1
    return r
def encode(data):
    m=len(data)
    r=redundant_bits(m)
    Print("Number of data bits:",m)
    Print("Number of redundant bits:",r)
    n=m+r
    h=['0']*(n+1)
    j=0
    for i in range(r):
        P=2**j
        Parity=0
        for j in range(1,n+1):
            if j&P:
                Parity^=int(h[j])
        h[P]=str(Parity)
    Print("Redundant bit Positions:",[2**i for i in range(r)])
    Print("Hamming code:","".join(h[1:]))
    return "".join h[1:])
def check(h):
    n=len(h)
    r=0
    while 2**r<=n
        r+=1
    error=0
    Print("In Parity Checking:")
    for i in range(r):
        P=2**j
        Parity=0
```

```
        for j in range (1, n+1):
        if j & p:
            parity ^= int(h[j-1])
        print ("Position", p, ":", parity)
    if parity != 0:
        error += p
    if error == 0:
        print ("\n No error detected")
    else:
        print ("Error has occured")
        print ("Error Position :", error)
        if h[error-1] == '0':
            h[error-1] = "1"
        else
            h[error-1] = '0'
        print ("error corrected")
    print ("Corrected Hamming code :", "".join(h))
    return h
    def remove_parity(h):
        data = ''
        for i in range (1, len(h)+1):
            if i & (i-1):
                data += h[i-1]
        return data
def binary_to_text(data):
    text = ''
    for i in range (0, len(data), 8):
        byte = data[i:i+8]
        text += chr(int(byte, 2))
    return text
text = input ("Enter data to transmit:")
binary = ''.join (format (ord(c), '08b') for c in text)
print ("\n Data:" text)
print ("Binary:", binary)
channel = encode (binary)
with open ("channel.txt", "w") as f:
    f.write (channel)
print ("\n Data saved in channel.txt")
```

Scanned with OKEN Scanner

```
with open ("channel.txt", "r") as f :
        received = f.read()
Print ("\n Received data :")
Print (received)
change = input ("\n Do you want to change the transmitted
                        Data ? (yes/no) :")
if change.lower == = "yes":
    Pos = int(input ("Enter position to change :"))
    if Pos >= 1 and Pos <= len (received):
        h=list (received)
        if h[Pos-1] == '0'
            h[Pos-1] = '1'
    else:
        h [Pos-1] = '0'
        received = ''.join(h)
        Print ("changed Data :")
        Print (received)
    else :
        Print ("Invalid Position")
        exit()
hamming = list (received)
hamming = check (hamming)
data = remove - Parity (hamming)
Print ("\n Data after removing redundant bits :")
Print (data)
output = binary - to - text (data)
  Print ("\n output :", output)
```

INPUT : Enter data to transmit : HI

Do you want to change the transmitted data ?

(yes/no) : no

OUTPUT : Data : HI

Binary : 0100100001001001

No. of data bits : 16

No. of redundant bits : 5

Redundant bit Positions : [1, 2, 4, 8, 16]

Hamming code : 10011001100001001001

Data saved in channel.txt

Scanned with OKEN Scanner

Received Data :

10011001100000100100

Data after removing redundant bits :

01001000010001100

RESULT ;

Thus the Program is implemented using Python for error correction & error detection using hamming code was executed successfully